

Rule of Zero, STL Контейнери и Уни указатели в C++

Семинар 10
Серхан

RAII - ресурсите се заделят в конструктора, освобождава се в деструктора (извиква се автоматично винаги)

Собственост - кой обект е отговорен за живота на ресурс;
кой трябва да го освободи

Smart pointer - клас, който обвива указател и управлява живота на обект чрез RAII

Rule of Zero - клас, чиито член-данни сами управляват ресурсите си,
не се пише голяма шестича

Reference counting - броят се колко указателя сочат към обект в heap.

Проблеми с ръчното new/delete:

○ забравено delete → паметта не се освобождава

○ delete на вече изтрият указател

○ използване на указател след delete

○ неясен собственик

Решението: STL класове и smart pointers, които прилагат RAII

std::string - STL клас за управление на текстови низове

- вътрешно управлява динамична памет
⇒ не се нуждае от Голяма шестича

std::vector<T> - динамичен масив, расте автоматично при нужда,
управлява паметта сам

Готови алгоритми от <algorithm> - работят чрез begin() и end(),
не знаат нищо за контейнера

• std::sort

• std::stable_sort - запазва относителния ред на елементи
с еднаква стойност

• std::find - първото срещане на стойност; връща итератор

- `std::find-if` - търси първия елемент, за който дадено условие е вярно въреща итератор
- `std::transform` - прилага функция върху всички елементи и записва резултата в друг диапазон (или същия)

Rule of Zero - ако член-данните сами управляват ресурсите, компилаторът генерира всички специални функции правилно - не пишем голяма шегица

Rule of Five	Rule of Zero
- при ръчна динамична памет (<code>char*</code>, <code>int*</code>) - трябва ръчно (<code>~T()</code>, <code>T(const T&)</code>, <code>T& op = (const T&)</code>, <code>T(T&&)</code>, <code>T& op = (T&&)</code>) - повече код, повече потенциални грешки	- при <code>std::string</code>, <code>std::vector</code>, <code>unique_ptr</code> - компилаторът генерира всичко правилно - по-малко код, по-безопасен, по-лесен за поддръжка

`std::pair<T1, T2>` - контейнер за два свързани обекта

`std::optional<T>` - представя стойност, която може да съществува или да липсва

Умни указатели

- клас, който обвива суров указател и прилага RAII

`std::unique_ptr<T>` - има един собственик, не се копира -

- създаване `auto p1 = std::make_unique<int>(42);` само се мести
`auto obj = std::make_unique<Student>("Иван", 20);`

• копирането е забранено
разрешено преместването

~~`auto p2 = p1;`~~

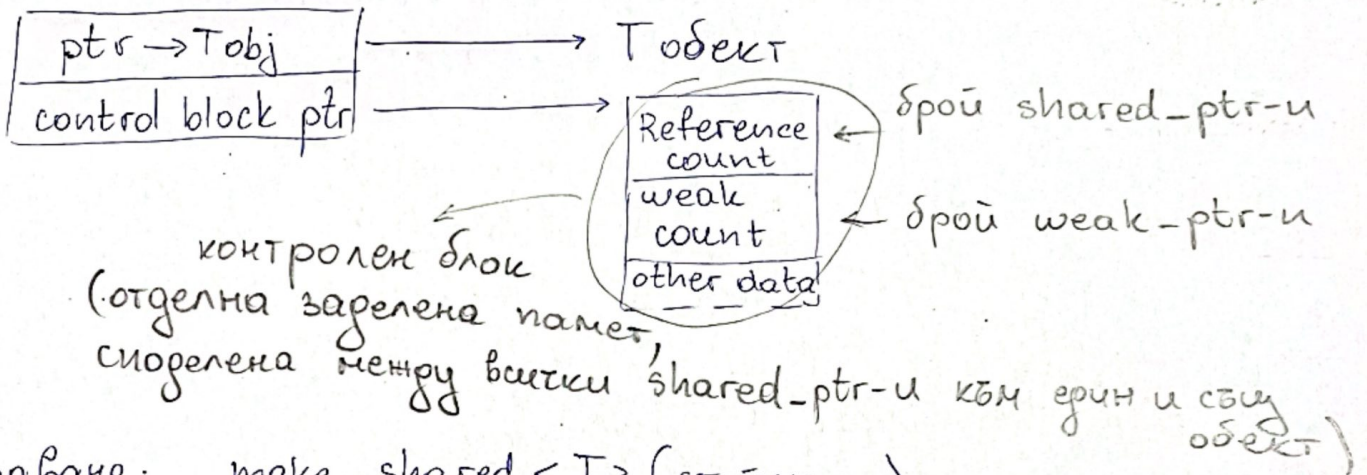
`auto p3 = std::move(p1);` ✓

↓
прехвърля собствеността и вече `p1 = nullptr;`

std::shared_ptr - споделена собственост (много shared_ptr-а могат да сочат към един обект)

- вътрешно съдържа два указателя:

`std::shared_ptr<T>`



• създаване: `make_shared<T>(стойности);`

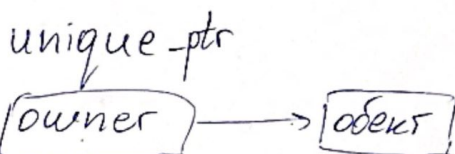
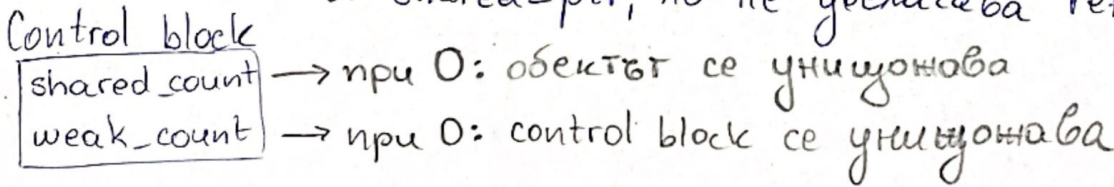
• споделяне: копирането увеличава броя

• при изтриване: броят се намалява

• проблем: Ако два обекта ^{се} сочат взаимно чрез shared_ptr, броят никога не стига 0 $\Rightarrow$ не се унищожават

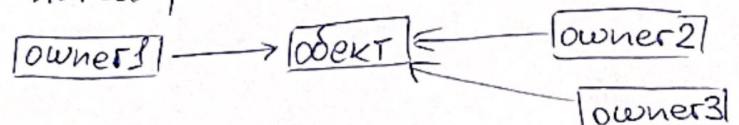
решение: weak_ptr

std::weak_ptr - непритежаващ указател; наблюдава обект, управляван от shared_ptr, но не увеличава reference count-а



- един собственик
- move-only

shared_ptr



- много собственици
- копира се
- reference counting

Кога кое да изберем:

- ① `unique_ptr` → обектът има 1 собственик
→ фабрични функции (Factory)
по-бърз
от
`shared_ptr` → при собственост, която се прерава върнат (move)
→ по поразбиране
- ② `shared_ptr` → обектът се споделя между много структури
по-бавен → не знаем живота на собствениците
от
→ дъщерни структури
`weak_ptr` → cache - обектът живее докато има потребители
- ③ `weak_ptr` → наблюдение без собственост
→ ~~приключва~~ прекъсване на циклични зависимости
→ observer pattern
→ cache - проверява дали обектът съществува